Reviewer Pack — Minimal Tropical Hodge Diamond Width and Communication Compl…

Entry 3b7e8b47e37d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 07:34:56 UTC

Minimal Tropical Hodge Diamond Width and Communication Complexity Rank Correlation

Entry ID: `3b7e8b47e37d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 07:34:56 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Hodge Theory) **Field B** (complexity object): Communication Complexity

Statement:

> For every circuit C with n inputs and m gates, the minimal tropical Hodge diamond width (THDW) of its associated Tseitin formula φ_C is linearly correlated with its communication complexity rank (CCR), such that THDW(φ_C) = Ω(CCR(φ_C)).

Rationale (proposer's reasoning):

> Tropical Hodge theory provides a geometric interpretation of the structure of complex algebraic varieties, and its invariants might capture non-trivial aspects of computational processes. The communication complexity rank measures the minimum number of bits needed to communicate a function's output between two parties. A correlation between these two could reveal deeper connections between geometric structures and information processing.

Taxonomy category: `geometric_information_theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0261e0922ec8874b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given circuit C, if the correlation coefficient between THDW(φ_C) and CCR(φ_C) is greater than or equal to 0.8 for all seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical Hodge diamond width" AND "communication complexity rank" - "Tseitin formula" IN Tropical Geometry AND communication complexity - "minimal tropical Hodge diamond width" correlative with communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 1:
            return [[0, 0], [1, 1]]
        else:
            subcircuits = [generate_circuit(n // 2) for _ in range(2)]
            circuit = []
            for i in range(n):
                if i % 2 == 0:
                    circuit.extend(subcircuits[0])
                else:
                    circuit.extend(subcircuits[1])
            return circuit

    def thdwidth(circuit):
        # Placeholder function to compute THD width
        return len(circuit)

    def ccr(circuit):
        # Placeholder function to compute CCR
        return len(circuit) // 2

    n = random.randint(5, 40)
    circuit = generate_circuit(n)
    thd_value = thdwidth(circuit)
    ccr_value = ccr(circuit)

    return {
```

```
        "metric_name": "THDW vs CCR",
        "metric_value": thd_value,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = [run_trial(seed) for seed in seeds]

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    print(f"RESULT: INCONCLUSIVE reason=undefined_mapping n_tested={len(seeds)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
RESULT: INCONCLUSIVE reason=undefined_mapping n_tested=30
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test code includes placeholder functions for computing the tropical Hodge diamond width (THDW) and communication complexity rank (CCR), which are not implemented. Without these implementations, it is impossible to assess whether the conjecture holds or not.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code includes placeholder functions for computing the tropical Hodge diamond width (THDW) and communication complexity rank (CCR), which are not implemented. As a result, it is impossible to assess whether the conjecture holds or not. | next: Implement the placeholder functions for THDW and CCR computation and rerun the test with actual data.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17063
2	propose	ollama_remote	glm4:latest	0	0	19998
3	preregistration	ollama_remote	glm4:latest	0	0	19671

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	8365
5	novelty	ollama_remote	glm4:latest	0	0	12028
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15721
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18098
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14839
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6793
10	critic	ollama_remote	glm4:latest	0	0	9990
11	judge	ollama_remote	glm4:latest	0	0	10011

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 152575 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/3b7e8b47e37d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3b7e8b47e37d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3b7e8b47e37d.tar.gz` (if generated)